# COMMON ACTIONS

**View online:** https://www.construct.net/en/make-games/manuals/construct-3/plugin-reference/common-features/common-actions

The following actions are common to several plugins.

## 3D rotation

Note only certain kinds of objects support 3D rotation.

### Set Euler rotation

Set the object's 3D rotation according to rotation around the X, Y and Z axes, in degrees. This approach of determining orientation is known as Euler rotation (or Euler angles). Note that changing the Euler rotation will also update the quaternion.

### Set quaternion

Set the object's 3D rotation according to a quaternion, given by its X, Y, Z and W components. Note that changing the quaternion will also update the Euler angles.

> *Quaternions are an advanced mathematical concept. In many cases it is easier to work with Euler angles.*

## Angle

Note angles in Construct start with 0 degrees facing right and increment clockwise.

### Rotate clockwise

Rotate the object a number of degrees clockwise.

### Rotate counter-clockwise

Rotate the object a number of degrees counter-clockwise.

### Rotate toward angle

Rotate the object a number of degrees toward another angle. If the object is close to the target angle it will not overshoot (it will end up exactly at the target angle).

### Rotate toward position

Rotate the object a number of degrees toward a position. If the object is close to facing the target angle it will not overshoot (it will end up exactly at the target angle).

---

**Set angle**

Set the object's angle in degrees, from 0 to 360. 0 degrees is facing right and angles increment clockwise.

---

**Set angle toward position**

Set the object's angle to face a position in the layout.

## Appearance

Not all objects support the actions relating to effects. For more information see Effects.

---

**Set blend mode**

Change the way the object blends with the background by altering the object's *Blend mode* property.

---

**Set color**

Change the *Color* property of the object, which applies a tint. Use the `rgbEx(r, g, b)` expression to set the color. To restore the original color of the object set a white color (i.e. `rgbEx(100, 100, 100)`).

---

**Set effect enabled**

Enable or disable one of the effects added to the object.

---

**Set effect parameter**

Change the value of one of the parameters for an effect added to the object. The parameter to change is specified by its zero-based index, i.e. 0 to change the first parameter, 1 to change the second parameter, and so on.

---

**Set opacity**

Set the object's opacity (or 'semitransparency'), from 0 (transparent) to 100 (opaque).

---

**Set sampling**

Set the object's sampling mode. For more information see the corresponding property in object instances.

### Set visible

Set the object visible or invisible (hidden). The *Toggle* option sets the opposite state, i.e. visible if the object is invisible and vice versa.

## Hierarchy

These actions are available for plugins that support the *scene graph* feature, allowing objects to be connected together so they move, rotate and scale as if they were one large object.

### Add child

Add another object as a child of this object, which becomes its parent. The relative differences between the objects are remembered at the time the action is used. Then as the parent moves, scales and rotates, the child will move, scale and rotate with it. A series of checkboxes allows selection of which properties are to be updated. For example ticking only the *X* and *Y* options will keep the child at the same relative position, but not change its angle. The *Destroy with parent* option can also be enabled to automatically destroy the child if its parent is destroyed.

### Remove child

Remove a child from this object, disconnecting it from the parent and no longer updating it as the parent changes. The child still keeps its own children, if it has any.

### Remove from parent

Remove this object from its parent if it has any, disconnecting it and no longer updating it as the parent changes. The object still keeps its own children, if it has any.

## HTML element

These actions are available for some plugins in the *Form controls* category, like Button and Text Input. These objects are HTML elements placed on top of the canvas.

### Set attribute

### Remove attribute

Add/set or remove an attribute on the HTML element for the form control. Attributes are part of the HTML standard and are documented on the Mozilla Developer Network.

> *Some attributes only need to exist to take effect and don't use a value. In this case, leave the value as an empty string.*

### Set CSS style

Set a CSS (Cascading Style Sheets) style on the HTML element for the form control. CSS is part of the HTML standard and is documented on the Mozilla Developer Network.

*Note that if the object has the Auto font size property enabled, any changes to the font-size property will be overridden.*

---

**Set enabled**

Enable or disable the control. When disabled, the control can no longer be interacted with. It also usually adjusts the appearance to indicate it is disabled, e.g. by greying out the control.

---

**Set focused**

**Set unfocused**

Focus or unfocus (also known as "blurring") the control. When focused, the control will receive keyboard input exclusively, and usually has an adjusted appearance to indicate active focus. When unfocused, the control will not respond unless directly clicked or touched, and the project will receive keyboard input instead.

---

**Set visible**

Set whether the HTML element is visible or hidden. The visibility can also be toggled, to switch the visibility state to the opposite (e.g. hide if showing, or show if hidden).

# Instance variables

---

**Add to**

**Subtract from**

Modify a number instance variable.

---

**Set**

Set a number or text instance variable.

---

**Set boolean**

Set a boolean instance variable, which can hold either a true or false value.

---

**Toggle boolean**

Toggle a boolean instance variable, which flips it from true to false or vice versa.

# Mesh distortion

These actions are available in objects that support the mesh distortion feature. This allows the object to be split in to a grid of points, and each point moved around individually to deform the appearance of the object. Moving mesh points also affects the object's collisions accordingly. See the mesh distortion examples in the Start Page for a demonstration.

### Set mesh size

Create a mesh using the given number of points horizontally (in columns) and vertically (in rows). At least 2 points must be used for both sizes for a mesh to be created. Use 0 for both to remove any existing mesh. Since the mesh is initialised with points in their default positions, no visual difference will be observed until a mesh point is altered. If there is an existing mesh, by default this action will reset all mesh points when the mesh size changes. To preserve the existing mesh points when resizing an existing mesh, check *Copy from old mesh*.

### Set mesh point

Alter one of the points on the mesh given by its zero-based column and row number. In *Absolute* mode the position and texture locations are set to the given values; in *Relative* mode the given values are added to their current values. The position is given as normalized co-ordinates in the range [0, 1] across the object box, i.e. 0.5 being in the middle. This allows the mesh to scale proportionately with the object size. Mesh points can be positioned outside the object box. The texture location is also given in normalized co-ordinates in the range [0, 1], but cannot go outside that range. Changing the texture location alters where on the source image corresponds to that mesh point, allowing for a different type of deformation. In absolute mode, texture positions of -1 leave the value unchanged. (In relative mode, use 0 to apply no change to the texture position, since -1 is a valid relative texture offset.) A Z position can also be specified for the mesh point, to move it in 3D. This works similarly to changing the Z position of the entire object, but only applying to a single mesh point. Note however the Z position is always interpreted as an absolute number regardless of the mode.

## Misc

### Change Tags

Change the instance's tags, using a space-separated string to specify a list of tags to change. The specified tags can either be added or removed from the instance, or replaced to set the instance's tags to just the provided ones. Note instance tags are case-insensitive.

### Destroy

Destroy the object instance so it no longer exists.

### Set from JSON

Set the state of this object from a string of data in JSON format. This must come from a prior use of the *AsJSON* expression.

---

**Signal**

Trigger *On signal* for the picked instances with the given tag. Any events paused with a *Wait for signal* action with the given tag will also be resumed once all the picked instances it is waiting on become signalled.

---

**Wait for signal**

Wait indefinitely until the *Signal* action is used with the same tag for all the currently picked instances. Other events continue to run in the meantime.

## Size & Position

---

**Move at angle**

Move the object a number of pixels at a given angle in degrees.

---

**Move forward**

Move the object a number of pixels forward at the object's current angle.

---

**Set width**

**Set height**

**Set depth**

**Set size**

**Set size 3D**

Set the object's current size in pixels. Note only 3D objects like 3D shape have the options to set depth or the 3D size, as 2D objects do not have a size on the Z axis.

---

**Set X**

**Set Y**

**Set Z**

**Set position**

**Set position 3D**

Set the object's current position in the layout, in pixels. The origin (0,0) is the top-left of the layout. The Y axis increments downwards and the Z axis increments towards the camera.

---

**Set position to another object**

Position the object at another object. It can also be positioned relative to an image point on the given object.

## Z Order

---

**Move to bottom**

**Move to top**

Position the object either at the bottom or top of its current layer.

---

**Move to layer**

Move the object to the top of a given layer, either by its name or zero-based index. If the object is already on the given layer this action has no effect.

---

**Move to object**

Move the object next to another object in the Z order. You can choose to place the object to be placed in front or behind another object. If the target object is on a different layer, the object will also be moved to the target object's layer and then Z ordered next to it.